

Discharge Summary AI

Backend Architecture & Entity-Relationship Diagram | Ashmit Verma

Project Name	Discharge Summary AI — MIMIC-IV Cardiology
Developer	Ashmit Verma
Repository	github.com/ashmit-verma24134/Integrated_Hospital [main branch]
Backend	Python FastAPI · Google Cloud SQL (PostgreSQL) · BigQuery ETL
AI Engine	Google Gemini 2.5 Flash (Pass 1 + Pass 2 prompting)
Frontend	Vanilla JS + Vite SPA — multi-screen role-based UI
API Port	8000 (main FastAPI) 7002 (data_server — BigQuery proxy)
Database	Cloud SQL (PostgreSQL) — 35+ tables Qdrant (vector store)
Data Source	MIMIC-IV v3.1 via Google BigQuery (physionet-data)
Document Date	June 2026

1. EXECUTIVE SUMMARY

Discharge Summary AI is an LLM-powered clinical document generation system designed for cardiology wards. It ingests structured patient data from MIMIC-IV (via Google BigQuery) or manually-uploaded CSV files, builds a comprehensive clinical context, and uses Google Gemini 2.5 Flash in a two-pass prompting pipeline to generate structured discharge summaries compliant with NABH standards.

Key capabilities: Multi-role authentication (Doctor / Admin Staff) with bcrypt password hashing and email-based password reset; CSV file upload for 18 clinical data types; BigQuery ETL pipeline for MIMIC-IV cardiology cohort (DCM, STEMI, HF patients); two-pass Gemini summarisation with NLI claim verification; NLI-based entailment scoring; doctor sign-off workflow; comprehensive audit log; Qdrant vector store for RAG; BM25 + dense hybrid retrieval.

2. TECHNOLOGY STACK

Layer	Technology	Details
API Framework	Python FastAPI	Async; 2946 lines; port 8000; Pydantic schemas
Data Server	FastAPI (data_server.py)	BigQuery proxy; port 7002; in-memory tab cache
AI Model	Google Gemini 2.5 Flash	Pass 1: structure+sections. Pass 2: refine+verify
Database	Cloud SQL (PostgreSQL)	Managed GCP; 35+ tables; WAL; ACID guarantees
Vector Store	Qdrant	Dense embeddings for RAG context retrieval
Embeddings	embedder.py	Sentence embeddings for RAG chunks
Indexer	BM25 (bm25_index.py)	Sparse retrieval; hybrid BM25+dense scoring
ETL	BigQuery (bulk_cardiology_etl.py)	MIMIC-IV cardiology cohort → Cloud SQL
Auth	bcrypt + session tokens	Bcrypt hashing; plain-text → bcrypt upgrade on login
Email	SMTP (Gmail)	Reset-password email via smtplib/MIMEMultipart
Frontend	Vanilla JS + Vite SPA	18+ screens; multi-role; nurse/doctor/admin/CMO
ORM/DB access	SQLAlchemy + raw SQL	cloud_sql_app_db.py; cloud_sql_db.py (pg engine)
Deployment	GCP Cloud Run / nginx	nginx.conf; deploy.sh; Docker-ready

3. BACKEND ARCHITECTURE

3.1 Service Architecture (Dual-Server)

The backend runs as TWO FastAPI processes:

- **main.py (port 8000)** — application API: auth, encounters, summaries, file uploads, Gemini generation, sign-off, audit log, settings.
- **data_server.py (port 7002)** — data proxy: reads clinical data from Cloud SQL (ap_* tables fetched from BigQuery), serves lazy tab data to the frontend, maintains an in-memory per-patient cache. Main server calls DATA_SERVER_URL internally via httpx.

3.2 Authentication Flow

Login: POST /api/auth/login → bcrypt.checkpw → session token. Password reset: forgot-password → generate token → store in app_reset_tokens → email via SMTP → /api/auth/reset-password validates token + expiry → bcrypt new hash → delete token. Plain-text passwords in DB are auto-upgraded to bcrypt on first successful login.

3.3 Discharge Summary Generation Pipeline (Two-Pass Gemini)

Pass 1: Build clinical context (build_clinical_context) → assemble structured prompt with NABH sections → Gemini 2.5 Flash generates full discharge summary draft.

Pass 2: Review pass — LLM verifies factual claims against source data; flags sections needing manual review. NLI entailment score saved to app_summaries.nli_score.

Prompt versions tracked (pass1_version, pass2_version) in app_summaries for A/B and error analysis.

3.4 CSV File Upload Pipeline

POST /api/encounters/{hadm_id}/ingest_file (18 file_type values supported) → CSV parsed → rows enriched (ICD titles via BigQuery, lab labels) → stored as JSONB in app_clinical_rows AND written to structured ap_* tables → data_server cache invalidated.

3.5 BigQuery ETL Pipeline

bulk_cardiology_etl.py / bigquery_mimic_loader.py: pulls MIMIC-IV cardiology cohort (DCM ICU patients filtered by ICD codes) from physionet-data.mimiciv_hosp / mimiciv_icu → inserts into Cloud SQL ap_* tables → updates active_patients.data_fetch_status. Advisory lock (pg_advisory_xact_lock) prevents double-fetch. Two rounds: hosp data first,

ICU data second.

3.6 RAG Context Assembly

chunker.py splits clinical data into text chunks → embedder.py creates dense vectors → stored in Qdrant. At summary time: section query → hybrid BM25 (bm25_index.py) + dense (qdrant_store.py) retrieval → top-k chunks assembled as context for Gemini prompt.

3.7 End-to-End Data Flow

Step	Layer	Action	Component
1	BigQuery	ETL: MIMIC-IV cardiology → Cloud SQL	bulk_cardiology_etl.py
2	Cloud SQL	Store in ap_* tables + active_patients	cloud_sql_db.py (get_engine)
3	Data Server (7002)	Serve lazy-tab data (labs, meds, vitals...)	data_server.py
4	Main API (8000)	Auth → encounter lifecycle management	main.py /api/auth, /api/encounters
5	CSV Upload	Admin uploads CSV → ingest → Cloud SQL	main.py /api/encounters/{id}/ingest_file
6	Context Builder	Aggregate clinical data → prose context	build_clinical_context()
7	RAG Retrieval	BM25+dense → relevant clinical chunks	bm25_index.py + qdrant_store.py
8	Gemini Pass 1	Full discharge summary generation (NABH)	Gemini 2.5 Flash API
9	Gemini Pass 2	Claim verification + NLI scoring	Gemini 2.5 Flash API
10	Sign-Off	Doctor reviews → signs → PDF generated	main.py /api/summaries/{id}/signoff
11	Audit Log	All actions logged with user_id + details	app_audit_log

4. DATABASE SCHEMA — APPLICATION TABLES

Database: Google Cloud SQL (PostgreSQL) · 35+ tables across 3 schema groups: App tables (workflow), Clinical data tables (ap_*), and Cardiology staging tables.

Table: app_users

System users — Doctors, Admin Staff, Billing Staff, Super Admin.

Primary Key	id (UUID, gen_random_uuid())			
Foreign Keys	None			
Column	Type	PK/FK	References	Notes
id	UUID	PK	-	Auto-generated
hospital_email	VARCHAR(255)	-	-	UNIQUE per (email, role)
role	VARCHAR(30)	-	-	Doctor Admin Staff Billing Staff Super Admin
full_name	VARCHAR(120)	-	-	
password_hash	TEXT	-	-	bcrypt hash; auto-upgraded on login
session_token	TEXT	-	-	Nullable; dummy token in demo
is_active	BOOLEAN	-	-	Default TRUE
created_at	TIMESTAMPTZ	-	-	
updated_at	TIMESTAMPTZ	-	-	Auto-updated trigger

Table: active_patients

Master patient hub — one row per registered hadm_id. Denormalised summary for fast list views.

Primary Key	id (UUID) + UNIQUE(hadm_id)			
Foreign Keys	assigned_doctor_id → app_users, signed_by → app_users, encounter_id → app_encounters, summary_id → app_summaries			
Column	Type	PK/FK	References	Notes
id	UUID	PK	-	
hadm_id	INTEGER	-	-	UNIQUE; MIMIC-IV admission ID
subject_id	INTEGER	-	-	MIMIC-IV patient ID
gender / anchor_age	CHAR/SMALL INT	-	-	Demographics snapshot
race / marital_status	VARCHAR	-	-	
admission_type	VARCHAR(50)	-	-	EMERGENCY ELECTIVE etc.
admit_time/discharge	TIMESTAMPTZ	-	-	
los_days	NUMERIC(6,2)	-	-	Length of stay
primary_diagnosis_*	VARCHAR/TEXT	-	-	Denormalised from ap_diagnoses
status	VARCHAR(30)	-	-	active→data_fetching→data_ready→...

data_fetch_status	VARCHAR(20)	-	-	pending fetching fetched partial failed
assigned_doctor_id	UUID	FK	app_users	
encounter_id	UUID	FK	app_encounters	
summary_id	UUID	FK	app_summaries	

Table: app_encounters

Discharge summary workflow case. One encounter per hadm_id.

Primary Key	id (UUID) + UNIQUE(hadm_id)			
Foreign Keys	hadm_id → active_patients(hadm_id), assigned_to → app_users			
Column	Type	PK/FK	References	Notes
id	UUID	PK	-	
hadm_id	INTEGER	FK	active_patients	ON DELETE CASCADE; UNIQUE
status	VARCHAR(40)	-	-	Pending Ingestion→Processing→Ready for Review→Signed Off
complexity	VARCHAR(20)	-	-	Low Medium High
assigned_to	UUID	FK	app_users	Assigning doctor/admin
version	INTEGER	-	-	Summary version counter
discharge_type	VARCHAR(20)	-	-	Standard LAMA DAMA Death Referral

Table: app_summaries

AI-generated discharge summary content + NLI verification metadata.

Primary Key	id (UUID) + UNIQUE(encounter_id)			
Foreign Keys	encounter_id → app_encounters, signed_by → app_users			
Column	Type	PK/FK	References	Notes
id	UUID	PK	-	
encounter_id	UUID	FK	app_encounters	ON DELETE CASCADE; UNIQUE
hadm_id	INTEGER	-	-	Denormalised for query
content	TEXT	-	-	Full generated summary (Markdown)
clinical_context	TEXT	-	-	Input context used for generation
nli_score	NUMERIC(5,4)	-	-	Entailment score 0.0–1.0
claim_verification_status	VARCHAR(30)	-	-	Needs Review Verified Manual
signed_by	UUID	FK	app_users	Nullable; doctor who signed
signed_at	TIMESTAMPZ	-	-	
pass1_version	TEXT	-	-	Prompt version tracking
pass2_version	TEXT	-	-	Prompt version tracking

Table: *app_uploaded_files*

File upload registry — tracks every CSV uploaded for an encounter.

Primary Key	id (UUID)			
Foreign Keys	hadm_id → active_patients, encounter_id → app_encounters, uploaded_by → app_users			
Column	Type	PK/FK	References	Notes
id	UUID	PK	-	
hadm_id	INTEGER	FK	active_patients	
encounter_id	UUID	FK	app_encounters	
file_type	VARCHAR(30)	-	-	labs meds notes diagnoses procedures ...
file_name	VARCHAR(255)	-	-	Original filename
file_size	BIGINT	-	-	Bytes
indexed_status	VARCHAR(20)	-	-	pending indexed failed
uploaded_by	UUID	FK	app_users	

Table: *app_clinical_rows*

Raw JSON blob storage for uploaded CSV rows (JSONB array per file).

Primary Key	id (BIGSERIAL) + UNIQUE(hadm_id, file_type, file_id)			
Foreign Keys	None (orphan-safe by design)			
Column	Type	PK/FK	References	Notes
id	BIGSERIAL	PK	-	
hadm_id	INTEGER	-	-	Not FK — orphan-safe for demo uploads
file_type	VARCHAR(30)	-	-	labs meds notes ...
file_id	UUID	-	-	Links to app_uploaded_files.id
rows	JSONB	-	-	Full CSV rows as JSON array

Table: *app_reset_tokens*

Single-use password reset tokens with 1-hour expiry.

Primary Key	token (TEXT)			
Foreign Keys	user_id → app_users			
Column	Type	PK/FK	References	Notes
token	TEXT	PK	-	32-byte URL-safe random
user_id	UUID	FK	app_users	ON DELETE CASCADE
expiry	BIGINT	-	-	Unix timestamp; token deleted on use
created_at	TIMESTAMPZ	-	-	

Table: *app_audit_log*

Append-only immutable audit trail for all system actions.

Primary Key	id (BIGSERIAL)			
-------------	----------------	--	--	--

Foreign Keys	user_id → app_users (nullable), hadm_id not FK			
Column	Type	PK/FK	References	Notes
id	BIGSERIAL	PK	-	
action	VARCHAR(80)	-	-	e.g. 'summary_generated', 'file_uploaded'
hadm_id	INTEGER	-	-	
user_id	UUID	FK	app_users	Nullable
details	JSONB	-	-	Action-specific payload
created_at	TIMESTAMPZ	-	-	Never updated

Table: app_settings

Singleton system settings (hospital name, NLI threshold, chunk count).

Primary Key	id = 1 (enforced by CHECK)			
Foreign Keys	None			
Column	Type	PK/FK	References	Notes
id	INTEGER	PK	-	Always 1 (singleton)
hospital_name / wing	VARCHAR	-	-	
nli_entailment_threshold	NUMERIC(4,3)	-	-	Default 0.5
k_retrieved_chunks	SMALLINT	-	-	Default 5
dense_retrieval_weight	NUMERIC(4,3)	-	-	Default 0.7
auto_notify_doctor	BOOLEAN	-	-	Default FALSE

Table: error_log

3-tier accuracy framework error logging for Tier 3 launch gate monitoring.

Primary Key	id (SERIAL)			
Foreign Keys	None			
Column	Type	PK/FK	References	Notes
id	SERIAL	PK	-	
hadm_id	VARCHAR(20)	-	-	Admission ID
error_tier	SMALLINT	-	-	1=minor, 2=moderate, 3=critical (launch gate)
nabh_section	VARCHAR(10)	-	-	s1...s15 NABH section
error_category	VARCHAR(50)	-	-	medication_dose diagnosis hallucination ...
ai_output	TEXT	-	-	AI-generated text
correct_value	TEXT	-	-	Ground truth
source_present	BOOLEAN	-	-	Was fact present in source data?

5. DATABASE SCHEMA — CLINICAL DATA TABLES (ap_*)

All ap_* tables are fetched from BigQuery MIMIC-IV v3.1 and stored in Cloud SQL. Every table has a FK to active_patients(hadm_id) ON DELETE CASCADE and a source_fetched_at timestamp. All inserts use INSERT ... ON CONFLICT DO NOTHING for idempotent re-fetching.

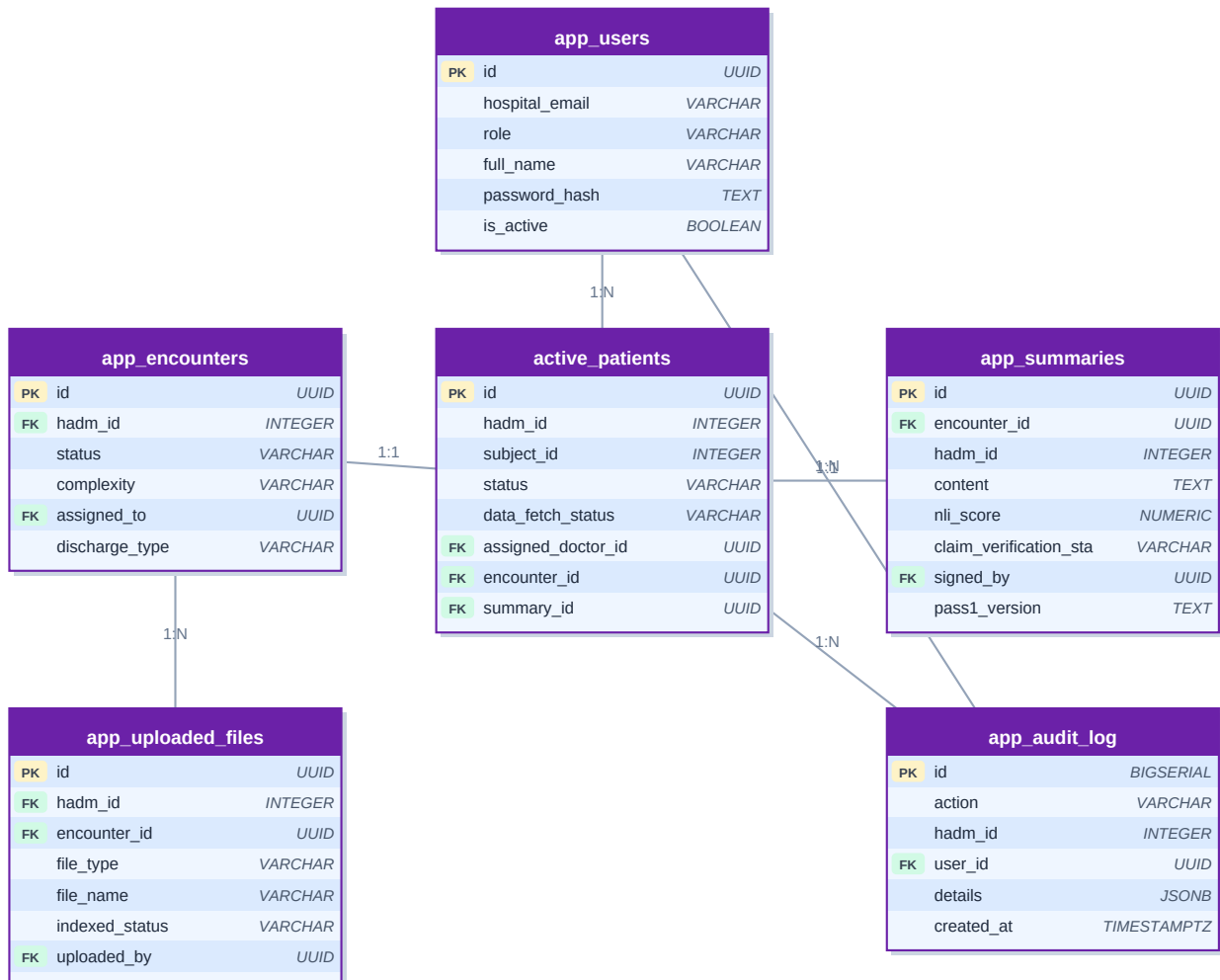
Table	Purpose	Primary Key	Key Columns
ap_admissions	Merged admissions + patients (demographics, diagnoses, procedures) → ap	hadm_id FK (to ap)	admittime, dischtime, gender, anchor_age
ap_diagnoses	ICD-9/10 diagnoses; computed is_primary BIGINT	BIGSERIAL id	seq_num, icd_code, icd_version, long_title, is_primary
ap_procedures	ICD procedure codes	BIGSERIAL id	seq_num, icd_code, icd_version, chartdate
ap_drgcodes	DRG billing codes (severity + mortality)	BIGSERIAL id	drg_type, drg_code, description, drg_severity
ap_labevents	High-volume lab results (~500-5000 rows/adm) → labevent_id BIGINT	labevent_id BIGINT	itemid, label, charttime, valuenum, flag, ref_range_*
ap_microbiologyevents	Culture results + antibiotic sensitivity	microevent_id	spec_type_desc, org_name, ab_name, interpretation
ap_prescriptions	Prescribed medications (prescriptions.csv)	BIGSERIAL id	drug, dose_val_rx, route, starttime, stoptime
ap_icustays	ICU stay records (parent of ICU sub-tables) → stay_id INT (MIMIC)	stay_id INT (MIMIC)	first_careunit, last_careunit, intime, outtime, los
ap_chartevents	Vitals + ICU monitoring (largest table per patient)	BIGSERIAL id	stay_id FK, itemid, charttime, value, valuenum, warning
ap_transfers	Patient movement through hospital wards	transfer_id INT	careunit, eventtype, intime, outtime
ap_services	Clinical service assignments (CMED, SURG, etc.)	BIGSERIAL id	transfertime, prev_service, curr_service
ap_pharmacy	Pharmacy dispensing + frequency for discharge → pharmacy_id BIGINT	pharmacy_id BIGINT	medication, frequency, route, status, starttime/stop
ap_poe	Physician orders (consults, radiology, discharge) → order_id VARCHAR(30)	order_id VARCHAR(30)	order_type, order_subtype, ordertime, order_status
ap_omr	Outpatient baseline vitals (OMR data)	BIGSERIAL id	chartdate, result_name, result_value
ap_procedureevents	ICU procedures (ventilation, CRRT, lines)	BIGSERIAL id	stay_id FK, itemid, label, starttime, endtime
ap_datatimeevents	ICU event timestamps (intubation, extubation, etc.)	BIGSERIAL id	stay_id FK, itemid, label, charttime, value (timestamp)
ap_inputevents	ICU IV fluids + infusions	BIGSERIAL id	stay_id FK, itemid, amount, rate, ordercategoryname
ap_outputevents	ICU fluid output (urine, drains)	BIGSERIAL id	stay_id FK, itemid, charttime, value

Cardiology Staging Tables (create_cardiology_tables.sql)

A parallel set of cardiology_* tables mirrors the ap_* schema for the legacy BigQuery bulk ETL pipeline. Tables include: cardiology_patients, cardiology_admissions, cardiology_diagnoses, cardiology_procedures, cardiology_labevents, cardiology_prescriptions, cardiology_icustays, cardiology_chartevents, cardiology_inputevents, cardiology_outputevents, cardiology_microevents, cardiology_emar, cardiology_pharmacy, cardiology_poe, cardiology_drgcodes, cardiology_services, cardiology_transfers + lookup tables: mimic_d_icd_diagnoses, mimic_d_icd_procedures, mimic_d_labitems, mimic_d_items.

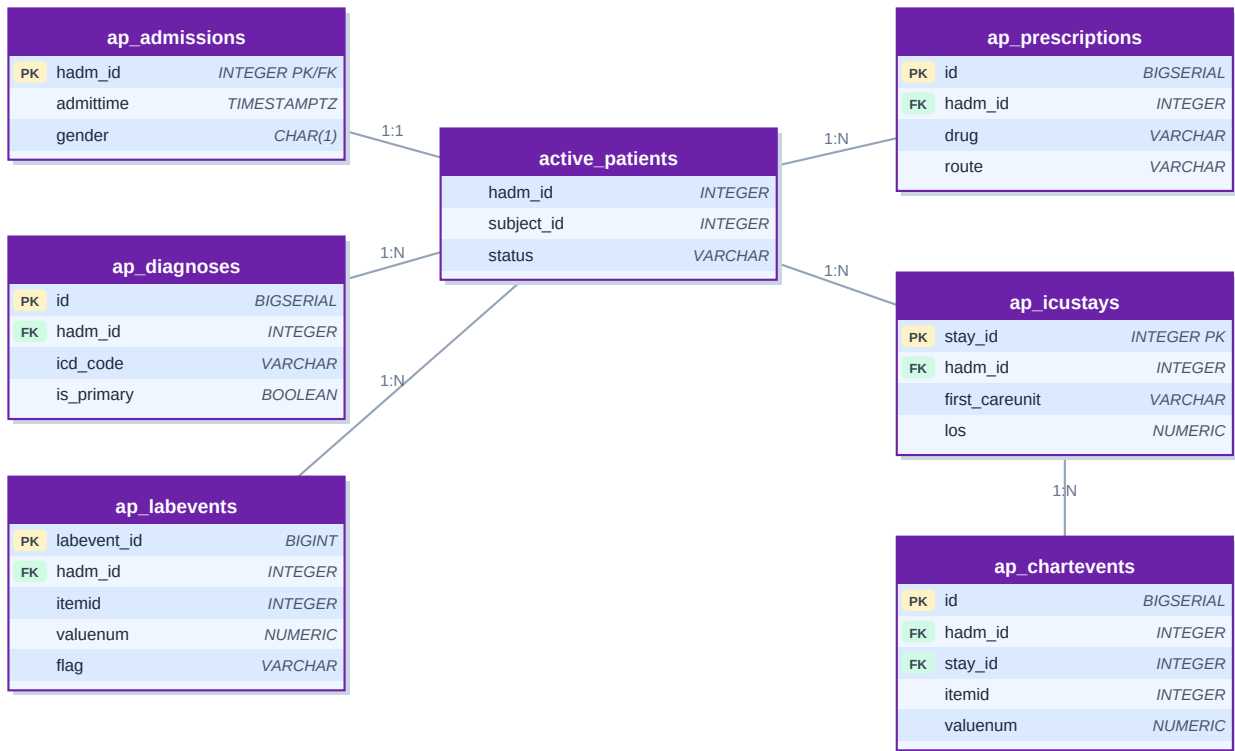
6. ENTITY-RELATIONSHIP DIAGRAM — APP TABLES

Core app-level relationships. Clinical data tables (ap_*) all FK to active_patients(hadm_id).



Clinical Data Table Relationships (ap_* tables)

All 18 clinical data tables (ap_admissions, ap_diagnoses, ap_labevents, etc.) hold a FK → active_patients(hadm_id) ON DELETE CASCADE. ICU sub-tables additionally FK to ap_icustays(stay_id).



7. API DOCUMENTATION SUMMARY

Authentication

Method	Endpoint	Description	Tables
POST	/api/auth/login	Bcrypt login; auto-detects role	app_users
POST	/api/auth/register	Create user (Doctor/Admin)	app_users
POST	/api/auth/forgot-password	Generate reset token + send email	app_reset_tokens
POST	/api/auth/reset-password	Validate token, set new bcrypt password	app_users, app_reset_tokens
POST	/api/auth/direct-reset	Force-set password (recovery only)	app_users

Dashboard & Encounters

Method	Endpoint	Description	Tables
GET	/api/dashboard	Stats: open cases, ready, signed this month	app_encounters, app_summaries
POST	/api/encounters	Create encounter for a hadm_id	app_encounters, active_patients
GET	/api/encounters	List encounters (filter by status/doctor)	app_encounters
GET	/api/active_patients	All loaded patients with encounter status	active_patients + app_encounters JOIN
POST	/api/seed/cardiology	Seed 10 DCM ICU encounters	app_encounters

File Upload & Ingest

Method	Endpoint	Description	Tables
POST	/api/encounters/{id}/ingest_file	Upload CSV (18 file_types)	app_clinical_rows + ap_* tables
GET	/api/encounters/{id}/uploaded_clinical_data	All uploaded rows	app_clinical_rows
DELETE	/api/uploaded_files/{file_id}	Delete file + its clinical rows	app_uploaded_files, app_clinical_rows
DELETE	/api/encounters/{id}/clinical_rows/{type}	Purge all rows by type	app_clinical_rows

Summaries & Sign-Off

Method	Endpoint	Description	Tables
GET	/api/summaries	List summaries with encounter/doctor info	app_summaries + app_encounters
POST	/api/patient/{id}/generate	Run Gemini Pass1+Pass2 generation	app_summaries, active_patients
POST	/api/summaries/{id}/signoff	Doctor sign-off	app_summaries, app_audit_log

8. ARCHITECTURE REVIEW

Patterns	Dual-server microservice (main + data_server). Repository pattern via cloud_sql_app_db.py. Factory pattern for DB engine. Strategy pattern for retrieval (BM25 vs dense).
Strengths	Production-grade Cloud SQL with ACID guarantees, advisory locks, SELECT FOR UPDATE. bcrypt auth. Two-pass LLM with NLI scoring. Comprehensive audit log. NABH-compliant output sections. Error log with Tier 3 launch gate.
Bottlenecks	Single Gemini API call per patient (latency ~5-15s). In-memory data_server cache is per-process (lost on restart). No real JWT (dummy token). BigQuery ETL is synchronous per patient.
Scalability	Cloud SQL scales vertically. Qdrant can be horizontally scaled. BigQuery ETL can be parallelised. Bottleneck is Gemini API rate limits. Add async task queue (Celery/Cloud Tasks) for generation.
DB Design	35+ tables, well-normalised with appropriate FKs, partial indexes, and UNIQUE constraints. Trigger-based updated_at. Concurrency patterns documented inline (advisory locks, SKIP LOCKED).
Improvements	Replace dummy session tokens with proper JWT. Add Celery for async Gemini generation. Implement Redis caching for data_server. Add rate limiting. Add WebSocket for real-time generation progress.

9. CONCLUSION

Discharge Summary AI is a sophisticated full-stack AI system that demonstrates strong engineering practices: production PostgreSQL with concurrency controls, a two-pass LLM pipeline with NLI-based hallucination detection, a comprehensive file upload and data ingestion pipeline supporting 18 clinical data types, bcrypt authentication with email-based password reset, and a role-based multi-screen frontend.

The system's 35+ table schema is well-designed for the problem domain: structured MIMIC-IV clinical data (18 ap_* tables) with FK cascade integrity, an application workflow layer (app_users, app_encounters, app_summaries, app_uploaded_files), and quality monitoring infrastructure (error_log with Tier 3 launch gate, app_audit_log). This is a production-grade architecture suitable for clinical deployment with the addition of real JWT tokens, async task queues, and Gemini rate-limit management.